```
Docker-
```

`Task 1. Setup a virtual machine using Ubuntu image.`

Task 1: Set Up a Virtual Machine Using an Ubuntu Image

Objective

To create and access a virtual machine using an Ubuntu operating system on AWS EC2.

Tools Required

- AWS Management Console
- Amazon EC2
- Ubuntu Server 26.04 LTS
- SSH / EC2 Instance Connect

Steps to Perform

Step 1: Log in to AWS

1. Open https://console.aws.amazon.com/.
2. Log in to the AWS Management Console.
3. Search for EC2.

Step 2: Launch an EC2 Instance

1. Open EC2 → Instances.
2. Click **Launch instances**.
3. Enter the instance name as `Ubuntu-VM`.
4. Select Ubuntu Server 26.04 LTS as the operating system image.
5. Select a suitable instance type.
6. Create or select a key pair.

7. Configure network settings and allow SSH access from your IP address.
8. Click **Launch instance**.

Step 3: Connect to the Virtual Machine

1. Wait until the instance state shows Running.
2. Select the instance.
3. Click Connect.
4. Use EC2 Instance Connect if available.
5. Confirm that the terminal opens successfully.

Step 4: Verify the Operating System

Run:

cat /etc/os-release

This command displays the operating system name and version.

Step 5: Verify the Logged-in User

Run:

whoami

Output:

ubuntu

This confirms that the current user is `ubuntu`.

Step 6: Verify Administrator Access

Run:

sudo whoami

Expected output:

root

This confirms that the user can execute commands with administrator privileges.

Real-Time Purpose

A virtual machine is a software-based computer that runs on physical cloud infrastructure.

In real-time projects, companies use AWS EC2 virtual machines to:

- Host websites and applications.
- Install DevOps tools such as Docker and Jenkins.
- Run application builds and tests.
- Create development, testing, and production environments.
- Deploy applications without purchasing physical servers.

How It Works

1. AWS provides physical computing infrastructure.
2. EC2 creates a virtual machine using the selected Ubuntu image.
3. AWS assigns the instance networking and storage resources.
4. The user connects to the instance through a supported remote-access method.
5. The user installs software and runs commands on the Ubuntu operating system.

Example: A DevOps engineer creates an Ubuntu EC2 instance, installs Docker, and runs an application inside a Docker container.

Result

Successfully launched an AWS EC2 virtual machine using Ubuntu Server 26.04 LTS and verified access to the operating system through the terminal.

```
The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-23-83:~$ dudo su
Command 'dudo' not found, did you mean:
  command 'udo' from deb udo (6.4.1-9)
  command 'sudo' from deb sudo (1.9.17p2-1ubuntu3)
  command 'sudo' from deb sudo-rs (0.2.13-0ubuntu1)
Try: sudo apt install <deb name>
ubuntu@ip-172-31-23-83:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
ubuntu@ip-172-31-23-83:~$ whoamiwhoami
whoamiwhoami: command not found
ubuntu@ip-172-31-23-83:~$ whoami
ubuntu
ubuntu@ip-172-31-23-83:~$ sudo whoami
root
```

Task 2: Install Docker on an Ubuntu Virtual Machine

Objective

To install Docker Engine on an Ubuntu virtual machine and verify that Docker can successfully run a container.

Tools Required

- AWS EC2 Ubuntu Virtual Machine
- Ubuntu Terminal

- Docker Engine
- Docker Hub

Steps to Perform

Step 1: Update Ubuntu Packages

Command:

sudo apt update

Purpose: Updates the list of available software packages in Ubuntu.

Step 2: Install Docker

Command:

sudo apt install docker.io -y

Purpose: Installs Docker Engine from the Ubuntu package repository.

Step 3: Start the Docker Service

Command:

sudo systemctl start docker

Purpose: Starts the Docker service so it can create and manage containers.

Step 4: Enable Docker at Startup

Command:

sudo systemctl enable docker

Purpose: Configures Docker to start automatically when the virtual machine boots.

Step 5: Verify Docker Installation

Command:

sudo docker --version

Output:

Docker version 29.1.3, build 29.1.3-0ubuntu4.1

Purpose: Confirms that Docker is installed.

Step 6: Test Docker Using Hello World

Command:

sudo docker run hello-world

Output:

Hello from Docker!
This message shows that your installation appears to be working correctly.

Purpose: Tests whether Docker can download an image, create a container, and execute it successfully.

Real-Time Purpose

Docker is widely used in DevOps to package applications and their dependencies into containers.

In real-time projects, Docker helps teams:

- Run applications in isolated containers.
- Maintain consistent environments across development, testing, and production.
- Deploy applications more easily.
- Integrate container builds and deployments with Jenkins CI/CD pipelines.
- Run services such as databases, web servers, and application servers.

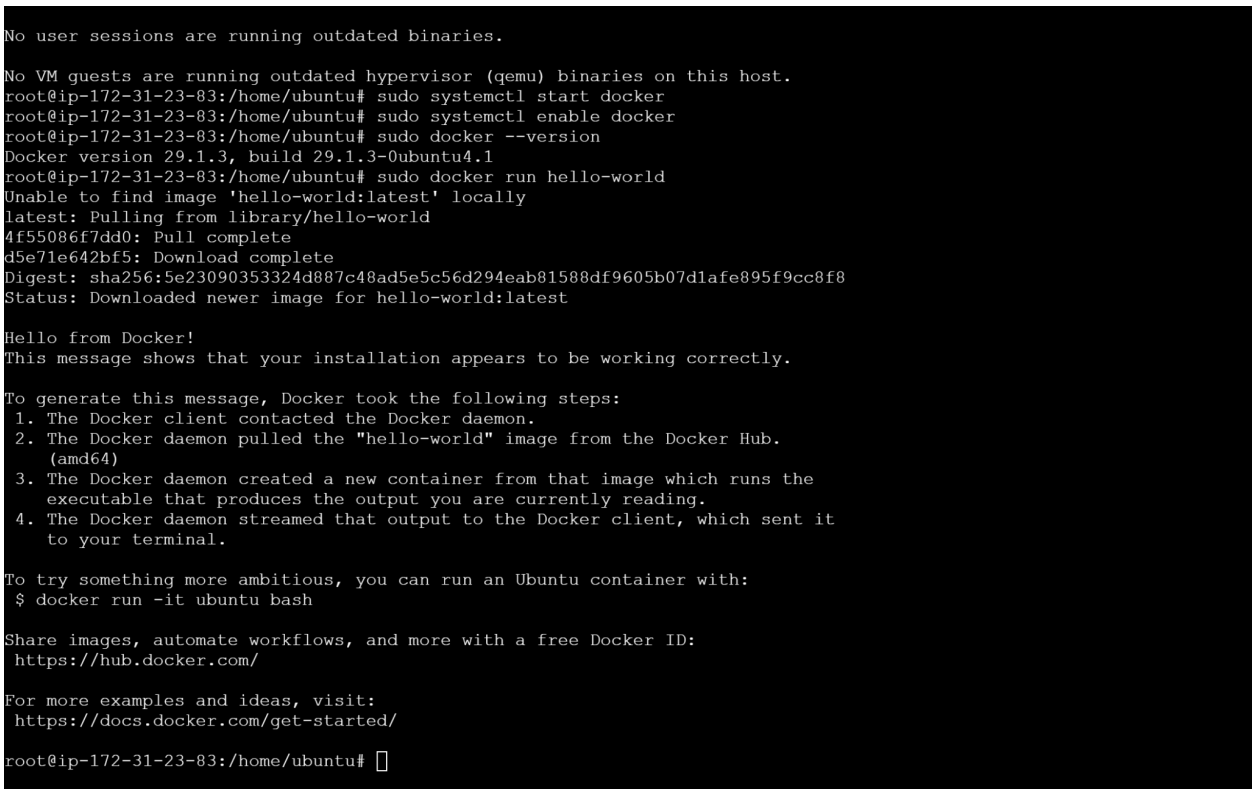
How It Works

1. The user runs a Docker command in the terminal.
2. The Docker client communicates with the Docker daemon.
3. The daemon downloads the required image from Docker Hub if it is not available locally.
4. Docker creates a container from the image.
5. The container executes the application or command.
6. The output is displayed in the terminal.

Example: When `docker run hello-world` is executed, Docker downloads the Hello World image, creates a container, runs the test program, and displays the success message.

Result

Successfully installed Docker Engine version 29.1.3 on the Ubuntu EC2 virtual machine and verified its functionality by running the Hello World container.



Task 3: Pull 5 Docker Images

Objective

To download five Docker images from Docker Hub: Nginx, Apache Tomcat, Ubuntu, Jenkins, and SonarQube.

Tools Required

- Ubuntu EC2 Virtual Machine
- Docker Engine
- Docker Hub
- Ubuntu Terminal

Steps to Perform

Step 1: Pull the Nginx Image

Command:

docker pull nginx

Purpose: Downloads the Nginx web server image, which can be used to host websites and serve web content.

Step 2: Pull the Apache Tomcat Image

Command:

docker pull tomcat

Purpose: Downloads the Apache Tomcat image, which is used to run Java web applications.

Step 3: Pull the Ubuntu Image

Command:

docker pull ubuntu

Purpose: Downloads the Ubuntu Linux image, which can be used to create containers for Linux-based applications and commands.

Step 4: Pull the Jenkins Image

Command:

docker pull jenkins/jenkins:lts

Purpose: Downloads the Jenkins Long-Term Support image, which can be used to run Jenkins for continuous integration and continuous delivery (CI/CD).

Step 5: Pull the SonarQube Image

Command:

docker pull sonarqube:lts-community

Purpose: Downloads the SonarQube Community Build image, which is used to analyze source code quality and identify bugs and code issues.

Step 6: Verify the Downloaded Images

Command:

docker images

Purpose: Displays the Docker images downloaded and stored locally on the virtual machine.

Verify that the following repositories appear in the output:

1. `nginx`
2. `tomcat`
3. `ubuntu`
4. `jenkins/jenkins`
5. `sonarqube`

Real-Time Purpose

Docker images help DevOps engineers run applications and tools without manually installing all their software dependencies on the host operating system.

In real-time projects:

- Nginx serves websites and can act as a reverse proxy.
- Tomcat runs Java web applications.
- Ubuntu provides a Linux environment for containerized tasks.
- Jenkins automates software builds, testing, and deployments.
- SonarQube analyzes source code quality.

How It Works

1. The user executes the `docker pull` command.
2. The Docker client sends the request to the Docker daemon.
3. The Docker daemon contacts Docker Hub.
4. The required image layers are downloaded to the local machine.
5. The images are stored locally and can be used to create containers.
6. The `docker images` command lists the downloaded images.

Note: Pulling an image downloads a template; it does not automatically start a container.

Result

Successfully downloaded five Docker images—Nginx, Apache Tomcat, Ubuntu, Jenkins, and SonarQube—from Docker Hub and verified them using the `docker images` command.

```
root@ip-172-31-23-83:/home/ubuntu# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
f7e1db57b58b: Pull complete
ecc510c1e359: Pull complete
04bdbc7ea6b5: Pull complete
807ab930f3a0: Pull complete
1efc53192927: Pull complete
947d22046304: Pull complete
8020170dc65f: Pull complete
a1b85ab28c90: Download complete
c329293b1d90: Download complete
Digest: sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
root@ip-172-31-23-83:/home/ubuntu# docker pull tomcat
Using default tag: latest
latest: Pulling from library/tomcat
4f4fb700ef54: Pull complete
4e07a0f12b2c: Pull complete
06ad70e463aa: Pull complete
5dec61325458: Pull complete
f39241fdc590: Pull complete
6d27111e20e8: Pull complete
130739ca07b1: Pull complete
a61c6f59ba30: Pull complete
abd8fb28c7b0: Download complete
e142b487e72c: Download complete
Digest: sha256:bb7c0c78078841047e2d475acce833ef337cb0007ca7c7e4c9ff9058d9019801
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
root@ip-172-31-23-83:/home/ubuntu# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
8f70d2bfe91a: Download complete
```

```
4f4fb700ef54: Pull complete
5d5a1fad7028: Pull complete
c7ad1fe61e07: Pull complete
60d98d907669: Pull complete
1df735f481ad: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
eb27e3a98da1: Pull complete
600f318704dc: Download complete
ce2bd68be4fc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
docker.io/library/sonarqube:lts-community
root@ip-172-31-23-83:/home/ubuntu# docker images

IMAGE                      ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest         5e2309035332       25.9kB         9.49kB   U
jenkins/jenkins:lts        a660310e39ad        720MB          246MB
nginx:latest               f9ea18bfa4fa        243MB         66.7MB
sonarqube:lts-community    f709975ab31d       1.02GB          398MB
tomcat:latest              bb7c0c780788        640MB          174MB
ubuntu:latest              f144425ff09b        162MB         45.6MB
root@ip-172-31-23-83:/home/ubuntu#
```

Task 4: Run an Nginx Container and Expose It on Port 81

Objective

To run an Nginx Docker container and access the Nginx web server through port 81.

Steps to Perform

Step 1: Start the Nginx container

docker run -d --name nginx-81 -p 81:80 nginx

Purpose: Creates and starts an Nginx container in the background and maps host port 81 to container port 80.

Step 2: Verify the running container

docker ps

Purpose: Checks whether the Nginx container is running and verifies the port mapping.

Step 3: Test the web server

curl http://localhost:81

Purpose: Sends an HTTP request to Nginx through port 81 and displays the returned HTML.

Step 4: Access Nginx from a browser

Open:

http://EC2-PUBLIC-IP:81

Purpose: Verifies that the Nginx web server is accessible externally, provided the EC2 security group permits the connection.

Real-Time Purpose

In real projects, Nginx is used to serve websites, deliver static content, and act as a reverse proxy. Docker port mapping makes a service inside a container accessible through a port on the host machine.

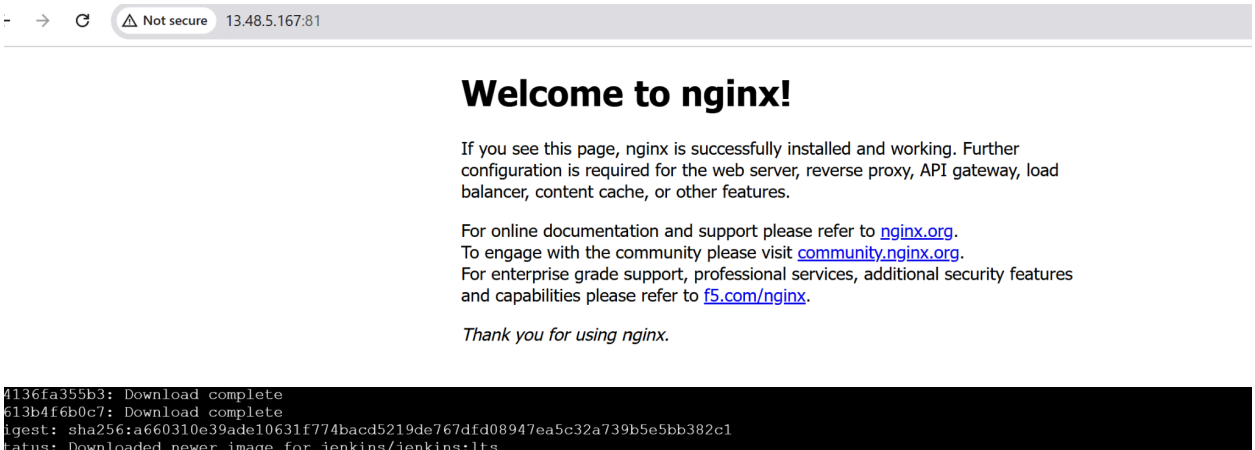
How It Works

1. Docker starts Nginx inside a container.
2. Nginx listens on port 80 inside the container.

3. Docker maps host port 81 to container port 80.
4. A request to the EC2 public IP on port 81 is forwarded to Nginx.
5. Nginx returns the web page to the browser.

Result

Successfully ran an Nginx container and exposed its web service on host port 81.




```
4136fa355b3: Download complete
613b4f6b0c7: Download complete
igest: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
tatus: Downloaded newer image for jenkins/jenkins:lts
ocker.io/jenkins/jenkins:lts
oot@ip-172-31-23-83:/home/ubuntu# docker pull sonarqube:lts-community
ts-community: Pulling from library/sonarqube
f4fb700ef54: Pull complete
d5a1fad7028: Pull complete
7ad1fe61e07: Pull complete
0d98d907669: Pull complete
df735f481ad: Pull complete
24a8b9e652f: Pull complete
3929ce9ef98: Pull complete
b27e3a98da1: Pull complete
00f318704dc: Download complete
e2bd68be4fc: Download complete
igest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
tatus: Downloaded newer image for sonarqube:lts-community
ocker.io/library/sonarqube:lts-community
oot@ip-172-31-23-83:/home/ubuntu# docker images

MAGE                     ID             DISK USAGE   CONTENT SIZE   EXTRA
ello-world:latest        5e2309035332       25.9kB         9.49kB   U
enkins/jenkins:lts       a660310e39ad        720MB          246MB
ginx:latest              f9ea18bfa4fa        243MB         66.7MB
onarqube:lts-community   f709975ab31d       1.02GB          398MB
omcat:latest             bb7c0c780788        640MB          174MB
buntu:latest             f144425ff09b        162MB         45.6MB
oot@ip-172-31-23-83:/home/ubuntu# docker run -d --name nginx-81 -p 81:80 nginx
a4b13c726debea1fe469a482d72d377d2ef21678bcfd8cb4345e3dcff0ecb03
oot@ip-172-31-23-83:/home/ubuntu# docker ps
ONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                     NAMES
a4b13c726de   nginx     "/docker-entrypoint.…"   12 seconds ago   Up 11 seconds   0.0.0.0:81->80/tcp, [::]:81->80/tcp   nginx-81
oot@ip-172-31-23-83:/home/ubuntu# curl http://localhost:81
!DOCTYPE html>
html>
head>
title>Welcome to nginx!</title>
```

Task 5: Delete the Apache Tomcat Image from Local Storage

Objective

To remove the Apache Tomcat Docker image from the local Docker image repository.

Tools Required

- Ubuntu EC2 Virtual Machine
- Docker Engine
- Ubuntu Terminal

Steps to Perform

Step 1: List Docker Images

Command:

docker images

Purpose: Displays the Docker images available locally and confirms whether the Tomcat image exists.

Step 2: Remove the Tomcat Image

Command:

docker rmi tomcat

Purpose: Deletes the local Tomcat image if no container is using it and no other image reference prevents removal.

Step 3: Verify Image Removal

Command:

docker images

Purpose: Confirms that the Tomcat image is no longer listed locally.

Real-Time Purpose

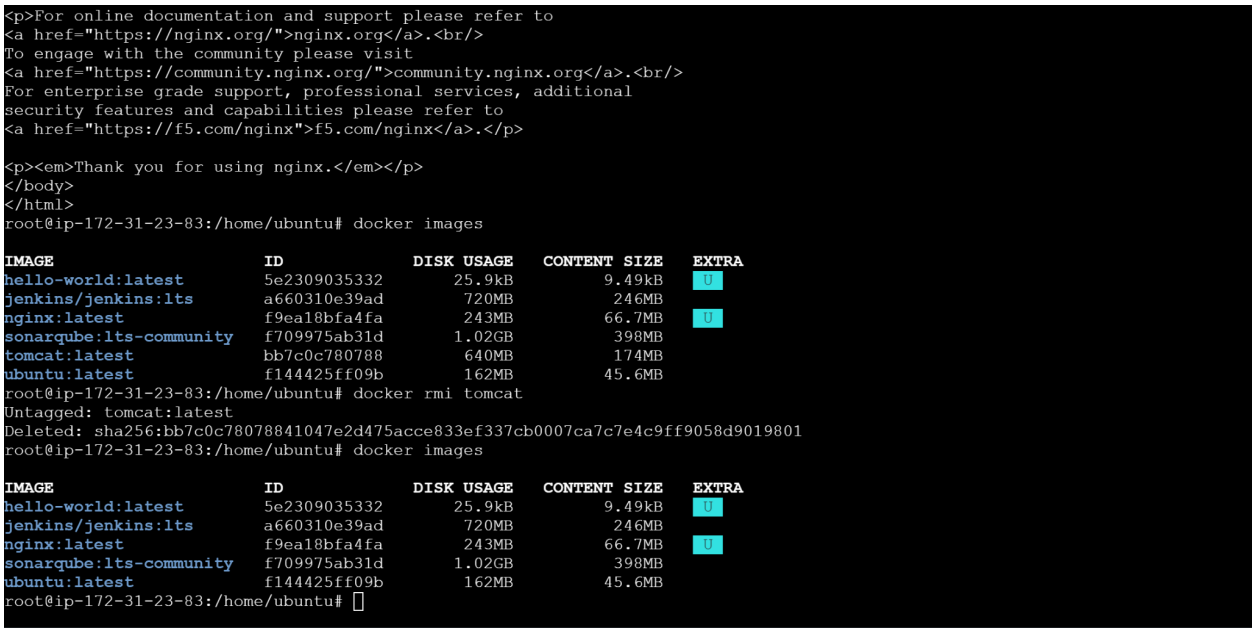
Removing unused Docker images helps free disk space on servers and keeps local image storage organized.

How It Works

1. Docker identifies the local Tomcat image.
2. Docker checks whether the image can be removed.
3. If it is not in use and removal is permitted, Docker deletes the image reference and associated image data that is no longer needed.
4. The user verifies the result using `docker images`.

Result

Successfully removed the Apache Tomcat Docker image from local storage and verified its removal.



Task 6: Inspect the Jenkins and SonarQube Docker Images

Objective

To inspect the Jenkins and SonarQube Docker images and understand their configuration and metadata.

Tools Required

- Ubuntu EC2 Virtual Machine
- Docker Engine
- Jenkins Docker Image
- SonarQube Docker Image

Steps to Perform

Step 1: Inspect the Jenkins Image

Command:

docker image inspect jenkins/jenkins:lts

Purpose: Displays detailed metadata about the Jenkins image, including its image ID, architecture, environment variables, and configuration.

Step 2: Inspect the SonarQube Image

Command:

docker image inspect sonarqube:lts-community

Purpose: Displays detailed metadata about the SonarQube image.

Step 3: Check the Image ID

For Jenkins:

docker image inspect --format='{{.Id}}' jenkins/jenkins:lts

For SonarQube:

docker image inspect --format='{{.Id}}' sonarqube:lts-community

Purpose: Displays the unique image ID for each image.

Real-Time Purpose

In DevOps projects, engineers inspect Docker images to understand image configuration, verify architecture, review environment settings, and troubleshoot container startup issues.

- Jenkins is used to automate software builds, tests, and deployments.
- SonarQube is used to analyze source code quality and identify code issues.

How It Works

1. The user runs the `docker image inspect` command.
2. Docker locates the specified image in local storage.
3. Docker reads the image metadata.
4. Docker displays the image configuration in JSON format.
5. The engineer uses the information to understand the image or troubleshoot problems.

Result

Successfully inspected the Jenkins and SonarQube Docker images and reviewed their metadata using Docker commands.





Task 7: Run a Jenkins Container and Execute a Sample Job

Objective

To run Jenkins using Docker and execute a sample Freestyle job.

Steps to Perform

1. Start Jenkins using the `jenkins/jenkins:lts` Docker image.
2. Map host port 8080 to container port 8080.
3. Verify the container using `docker ps`.
4. Retrieve the initial administrator password.
5. Open Jenkins through the EC2 public IP and port 8080.
6. Complete the initial setup and create an administrator account.
7. Create a Freestyle project named `Docker-Sample-Job`.
8. Add an Execute shell build step with sample `echo` commands.
9. Save the job and click Build Now.
10. Verify the output in Console Output.

Real-Time Purpose

Jenkins automates repetitive software development tasks such as building, testing, and deploying applications. Running Jenkins in Docker simplifies its installation and provides an isolated environment.

How It Works

Docker starts the Jenkins application inside a container. The user accesses Jenkins through a browser, creates a job, and triggers a build. Jenkins executes the configured commands and displays the results in the console.

Result

Successfully ran Jenkins in a Docker container and executed a sample Jenkins job.

```
2026-10-12 12:30:26.112+0000 [id=1]     INFO    o.e.j.s.handler.ContextHandler#doStart: Started oeje9n.ContextHandler$CoreContextHandler@5
80.1,/,b=file:///var/jenkins_home/war/,a=AVAILABLE,h=oeje9n.ContextHandler$CoreContextHandler$CoreToNestedHandler@79d9214d{STARTED}}
2026-10-12 12:30:26.127+0000 [id=1]     INFO    o.e.j.server.AbstractConnector#doStart: Started oejs.ServerConnector@3a543f31{HTTP/1.1, (h
80}
2026-10-12 12:30:26.130+0000 [id=1]     INFO    org.eclipse.jetty.server.Server#doStart: Started oejs.Server@2b9f74d0{STARTING}[12.1.13,st
2026-10-12 12:30:26.131+0000 [id=31]    INFO    winstone.Logger#logInternal: Winstone Servlet Engine running: controlPort=disabled
2026-10-12 12:30:26.253+0000 [id=30]    INFO    jenkins.model.Jenkins#<init>: Starting version 2.580.1
2026-10-12 12:30:26.335+0000 [id=39]    INFO    jenkins.InitReactorRunner$1#onAttained: Started initialization
2026-10-12 12:30:26.350+0000 [id=38]    INFO    jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2026-10-12 12:30:27.274+0000 [id=40]    INFO    jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2026-10-12 12:30:27.278+0000 [id=39]    INFO    jenkins.InitReactorRunner$1#onAttained: Started all plugins
2026-10-12 12:30:27.279+0000 [id=38]    INFO    jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2026-10-12 12:30:27.461+0000 [id=38]    INFO    jenkins.InitReactorRunner$1#onAttained: System config loaded
2026-10-12 12:30:27.463+0000 [id=40]    INFO    jenkins.InitReactorRunner$1#onAttained: System config adapted
2026-10-12 12:30:27.463+0000 [id=40]    INFO    jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2026-10-12 12:30:27.464+0000 [id=40]    INFO    jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
2026-10-12 12:30:27.507+0000 [id=54]    INFO    hudson.util.Retrier#start: Attempt #1 to do the action check updates server
2026-10-12 12:30:28.057+0000 [id=40]    INFO    jenkins.install.SetupWizard#init:
[LF]>
[LF]> *************************************************************
[LF]> *************************************************************
[LF]> *************************************************************
[LF]>
[LF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[LF]> Please use the following password to proceed to installation:
[LF]>
[LF]> 02d2c27cb6624e1fb963ab35eede8cbd
[LF]>
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]>
[LF]> *************************************************************
[LF]> *************************************************************
[LF]> *************************************************************

2026-10-12 12:30:31.867+0000 [id=37]    INFO    jenkins.InitReactorRunner$1#onAttained: Completed initialization
2026-10-12 12:30:31.888+0000 [id=30]    INFO    hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
2026-10-12 12:30:32.722+0000 [id=54]    INFO    h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.tasks.Mav
2026-10-12 12:30:32.722+0000 [id=54]    INFO    hudson.util.Retrier#start: Performed the action check updates server successfully at the a
root@ip-172-31-23-83:/home/ubuntu#
```

